

Caching Strategies Part 2

Prepared By: Dr. D. P. Singh

Read-Through Caching Strategy

- **Read-Through Cache** is a caching pattern where the **cache sits between the application and the database**.
- The **application reads only from the cache**, and the cache automatically fetches data from the database when there's a cache miss.

Cache Hit and Cache Miss(Redis)

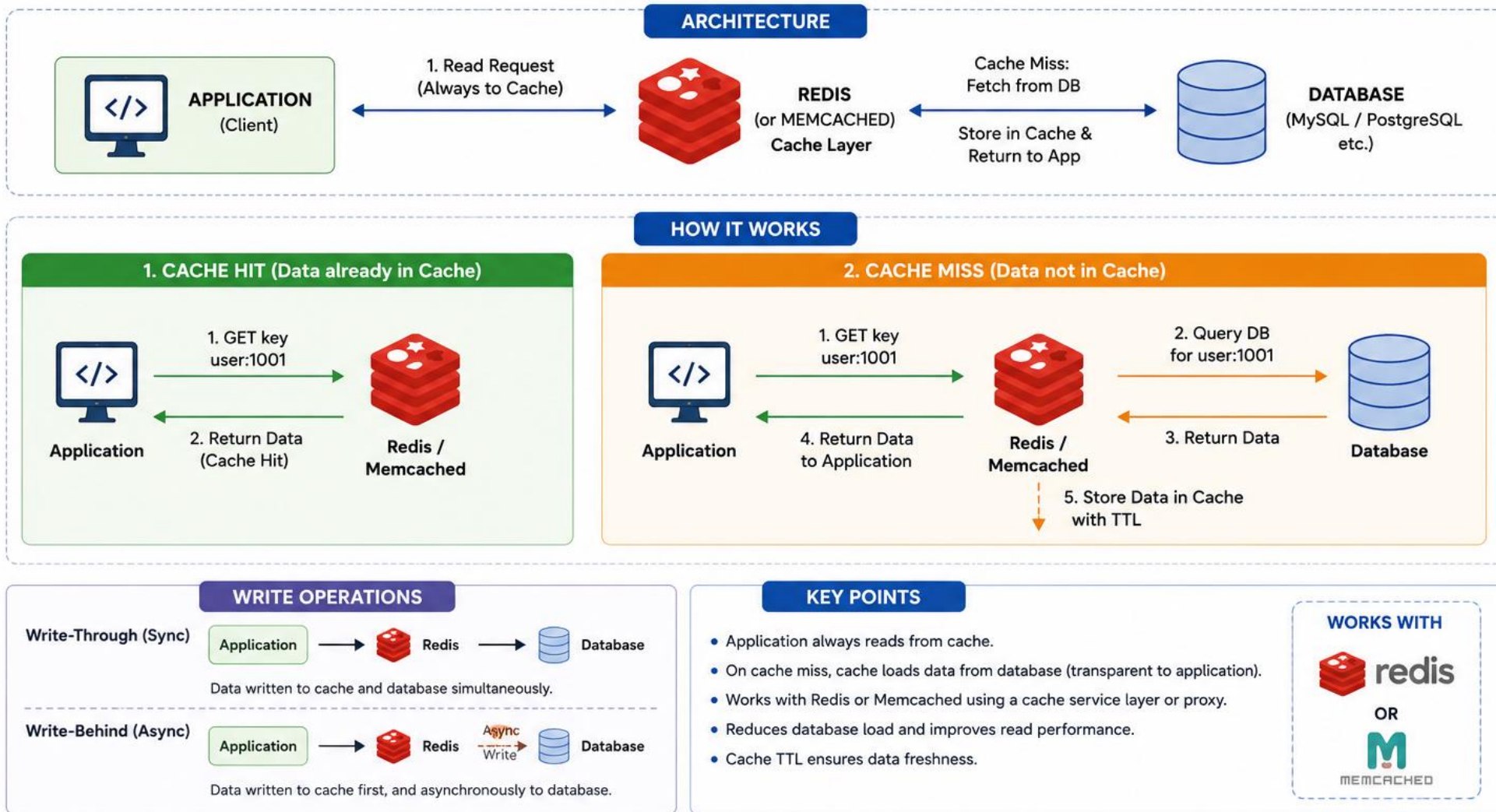
Application
|
GET user:1001
|
Redis
|
(Cache Hit)
|
Return Data

Application
|
GET user:1001
|
Redis
|
(Cache Miss)
|
Fetch from Database
|
Store in Cache
|
Return Data

Working

Read-Through Cache Pattern (Redis or Memcached)

The application reads only from the cache. On a cache miss, the cache automatically fetches data from the database, stores it, and returns it to the application.



Example: User requests /users/1001 → Cache checks Redis → If not found, Redis fetches from DB, stores it, and returns to user. Next time, it's a cache hit!

Advantages

1. Simpler Application Code

- Application always reads from cache
- No cache-miss handling in business code.

2. Centralized Cache Logic

- All caching behavior lives in one place.
- Benefits:
 - Easier maintenance
 - Consistent caching rules
 - Better observability

Advantages

contd...

3. Reduced Code Duplication

- Multiple services can use the same caching layer.

4. Better Developer Experience

- Developers don't need to remember:
 - Cache lookup
 - Database fallback
 - Cache population
- The **cache handles it automatically.**

Disadvantages

1. More Infrastructure Complexity

- Need a cache provider or middleware capable of:
 - Loading data from DB
 - Serialization
 - Error handling

2. Potential Cache Bottleneck

- All reads go through the cache layer.
- A cache outage affects all reads.

Disadvantages

contd...

3. Less Flexibility

- Application loses direct control over:
 - Cache population
 - Expiration strategies
 - Refresh policies

• 4. Cold Start Problem Still Exists

- After restart:
 - Database may experience spikes.

Cache Empty
↓
Many Requests
↓
Many DB Queries

Cache-Aside vs Read-Through

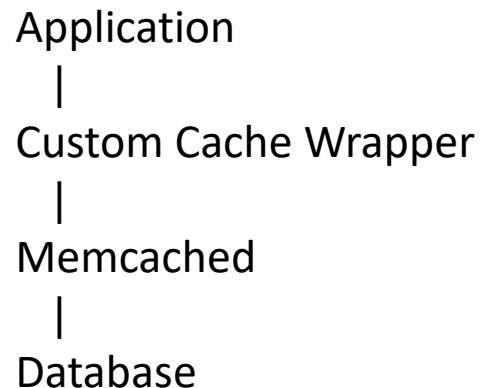
Feature	Cache-Aside	Read-Through
Database queried by	Application	Cache layer
Cache miss handling	Application	Cache
Complexity in app	Higher	Lower
Control	High	Medium
Infrastructure	Simpler	More complex
Redis native support	Easy	Requires wrapper/proxy

Redis and Memcached Support

- **Memcached**

- Memcached is a simple key-value cache and **does not natively support Read-Through caching**.

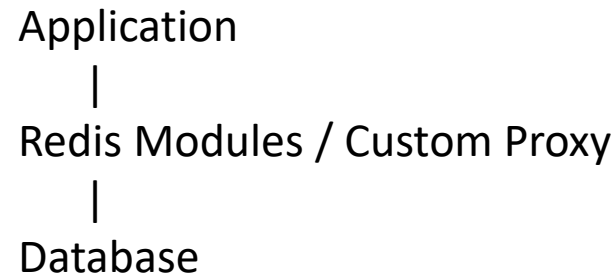
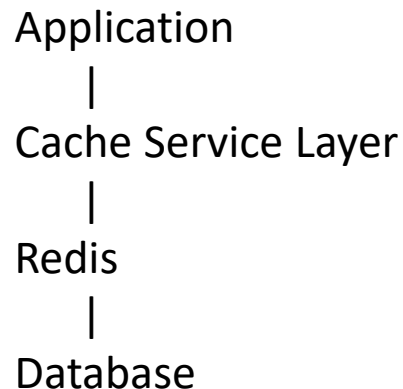
- To implement Read-Through:



Redis and Memcached Support

- **Redis**

- Open-source Redis also does **not provide built-in Read-Through** functionality.
- Common implementations:



- Some managed caching products built on Redis provide Read-Through capabilities, but standard Redis typically **requires application or middleware logic**.

When to Use Read-Through

- ✓ Large systems with many services
- ✓ Centralized caching platform
- ✓ Teams wanting simpler application code
- ✓ Consistent caching rules across services

When Cache-Aside Is Better

- ✓ Microservices architectures
 - ✓ Standard Redis/Memcached deployments
 - ✓ Need fine-grained cache control
 - ✓ Most web applications
- In practice, **Cache-Aside is by far the most commonly used pattern with Redis and Memcached**, while Read-Through is often used in enterprise caching platforms where a dedicated cache layer manages database access automatically.

- Thank You